HTML generation code properly, but initially there were no obvious problems. The code worked, so we shipped it!

A few months later, someone new started working on the app and immediately "helped" by renaming an innocuous spelling error in a class in the business layer. Unfortunately, the names of those classes had been used to generate our HTML code, so renaming the class caused the whole website to break in users' browsers! Suffice it to say that we made a concerted effort to apply the MVC pattern thereafter and ensure that we had a proper separation of concerns.

The examples shown in this chapter demonstrate the bulk of the Razor Pages functionality. It has additional features, such as the filter pipeline, which I cover in chapters 21 and 22, and I discuss binding models in greater depth in chapter 16, but the overall behavior of the system is the same.

Similarly, in chapter 19 I look at MVC controllers and explain why I don't recommend them over Razor Pages for server-rendered applications. By contrast, in chapter 20 I discuss how you can use the MVC design pattern when you're generating machine-readable responses using Web API controllers. The process is for all intents and purposes identical to the MVC pattern you've already seen.

I hope that by this point, you're sold on Razor Pages and their overall design using the MVC pattern. The page handler methods on a Razor Page are invoked in response to a request and select the type of response to generate by returning an `IActionResult`.

An aspect I've touched on only vaguely is how the `RoutingMiddleware` decides which Razor Page and handler to invoke for a given request. You don't want to have a Razor Page for every URL in an app. It would be difficult to have, for example, a different page per product in an e-shop; every product would need its own Razor Page! In chapter 14 you'll see how to define routes for your Razor Pages, how to add